BrainAI

ProSuite Industrial

PROSUITE INDUSTRIAL

PRODUKT-DOKUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 ÖFFENTLICHE KLASSIFIZIERUNG
Core Architecture & Development	**Technical Documentation**	Zugriffsstufe: Öffentlich / Beta

---

Revisionshistorie

Version	Datum	Autor	Status	Beschreibung
1.0.0	28.02.2026	S. Köhne	Stable	Initial Release: ProWB System & Integration Guide

1. Produktübersicht: Pro Web Builder (ProWB)

Der **ProWB** ist ein in reinem C geschriebener, statischer Site-Generator und Monolithen-Builder. Er wurde speziell entwickelt, um die kryptografischen WebAssembly-Module (ProED, ProChat, ProKey, ProHash) in einer absolut sicheren, isolierten und Zero-Trust-konformen Web-Umgebung zu kapseln.

Im Gegensatz zu klassischen Web-Frameworks (wie React oder Angular), die anfällig für Supply-Chain-Angriffe sind, erzeugt der ProWB eine einzelne, statische `index.html`. Alle CSS-Styles, JavaScript-Routinen und HTML-Ansichten werden während des Build-Prozesses in diesen Monolithen injiziert.

2. Die Verzeichnisstruktur (Source Tree)

Der ProWB erwartet ein striktes Ordner-Layout im Quellverzeichnis (`src_dir`). Jede Abweichung wird vom Builder ignoriert, um unautorisierte Datei-Injektionen zu verhindern.

```
prowb/src/
├── parts/      # HTML-Rahmengerüste (Header, Footer, Navigation)
├── css/        # Cascading Style Sheets (werden inline injiziert)
├── js/         # JavaScript-Logik & WASM-Bridges (werden inline inji
├── views/      # Native HTML-Ansichten (Dashboards, UI-Panels)
└── docs/       # Markdown-Handbücher (werden nativ geparst)
```

2.1 Das "00_ bis 99_" Sortier-System

Dateien in den Ordnern `views/`, `docs/`, `css/` und `js/` werden strikt alphabetisch eingelesen. Um die Ladereihenfolge (wichtig für CSS/JS) und die Reihenfolge der Navigations-Buttons zu steuern, nutzt ProWB einen numerischen Präfix:

- `00_vars.css` wird vor `99_theme.css` geladen.
- `00_view_home.html` erscheint in der Navigation ganz oben.

Sicherheits-Feature: Der Parser schneidet dieses Präfix beim Kompilieren intelligent ab. Aus `00_view_home.html` wird intern die saubere JavaScript-ID

`view_home` generiert.

3. Aufbau der HTML-Dateien (`/views` & `/parts`)

3.1 Die Rahmengerüste (`/parts`)

Diese Dateien bilden das Skelett des Monolithen:

- **`header.html`**: Beinhaltet den `<head>`-Bereich, Meta-Tags und die Eröffnung des `<body>`.
- **`footer.html`**: Schließt das Dokument sauber ab.
- **`nav.html`**: Enthält das Template für die Navigation. Der ProWB sucht hier nach zwei spezifischen Platzhaltern:
- `{{NAV_VIEWS}}`: Hier injiziert der C-Code automatisch die Buttons für alle Dateien aus dem `/views`-Ordner.
- `{{NAV_DOCS}}`: Hier injiziert der C-Code die Buttons für alle Markdown-Dateien aus dem `/docs`-Ordner.

3.2 Die UI-Ansichten (`/views`)

Da der Builder alle HTML-Dateien aus `/views` hintereinander in die `index.html` schreibt, müssen diese Dateien sich selbst als ausblendbare Blöcke definieren.

Zwingende Struktur für eine View-Datei (z. B. `10_view_prokey.html`):

```
<section id="view_prokey" class="view-content" style="display:none;">
    <h2>Hardware Entropy (ProKey)</h2>
    </section>
```

- **Wichtig:** Die `id` muss exakt dem Dateinamen (ohne das `00_` Präfix und ohne `.html`) entsprechen. Die `app.js` nutzt diese ID, um den Block beim Klick auf die Navigation sichtbar zu machen.

4. Vorbereitung der Markdown-Dokumente (`/docs`)

Dokumente im `/docs`-Verzeichnis (z.B. `90_Lizenz.md`) werden vom militärisch gehärteten `md_parser.c` verarbeitet.

4.1 Automatisches Wrapping

Im Gegensatz zu den HTML-Views müssen Sie bei Markdown-Dateien **keine** `<section>`-Tags manuell hinzufügen. Der Parser generiert automatisch den folgenden Rahmen um Ihr Dokument:

```
<section id="doc_Lizenz" class="view-content" style="display:none;">
    <div class="markdown-body">
        </div>
</section>
```

4.2 Best-Practices für ProWB-Markdown (GFM)

- **Überschriften:** Zwingend ein Leerzeichen nach der Raute setzen (`# H1`, `## H2`).
- **Sichere Dateinamen:** Verwenden Sie für `.md`-Dateien keine Sonderzeichen oder Leerzeichen (z. B. `05_Compliance_Guide.md`). Der Parser filtert gefährliche Zeichen zwar in Unterstriche (_) um, saubere Namen verhindern jedoch Darstellungsfehler.
- **Code-Blöcke:** Geben Sie die Sprache an (z. B. ```c), da der Parser dies in ` umwandelt, was später von Highlightern (PrismJS) genutzt werden kann.

5. Der Build-Prozess

Der ProWB wird über die Kommandozeile aufgerufen und erwartet drei Parameter:

1. **Source-Verzeichnis:** Der Pfad zum src-Ordner.
2. **Output-Verzeichnis:** Der Basis-Pfad, in dem das fertige Projekt abgelegt wird.
3. **Core-Name:** Der Name des zu ladenden WASM-Kerns (z. B. proedc).

Beispiel-Aufruf (Linux/Windows):

```
./prowb ./src/prowb ./dist proedc
```

- **Injection-Mechanismus:** Der Builder schreibt zusätzlich automatisch die Zeile <script>const PRO_WASM_CORE = 'proedc';</script> in den Output, damit die JavaScript-Logik weiß, welches Kryptografie-Modul geladen werden muss.

© 2026 Sascha Köhne	✉️ **Contact:** koehne83@googlemail.com	🛡️ **ProTU-Verified**
System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**